

```

from sklearn.model_selection import train_test_split
import numpy as np

features = [
    'season', 'yr', 'holiday', 'workingday', 'weathersit',
    'temp', 'atemp', 'hum', 'windspeed',
    'hr',
    'dayofweek', 'weekofyear', 'dayofyear',
    'is_weekend', 'is_rush_hour', 'is_night',
    'temp_diff', 'temp_hum', 'temp_wind',
    'mnth_sin', 'mnth_cos',
    'cnt_lag_1', 'cnt_roll_7'
]

# Target
y = np.log1p(df_model['cnt'])

# ✅ CORRECT FEATURE MATRIX
X = df_model[features]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(X_train.shape, X_test.shape)

(13898, 23) (3475, 23)

```

“In this part of project, I am preparing the data so that a machine learning model can be trained properly.”

“First, I imported some tools that help in machine learning. One tool is used to divide the data into training and testing parts, and another tool is used for mathematical calculations.”

“Next, I selected the important columns from the dataset. These columns are called **features**, and they are the inputs given to the machine.”

“These features include information like season, year, weather, temperature, humidity, wind speed, time of the day, whether it is a weekend or rush hour, and also past bike rental counts.”

“In total, I selected **23 important features** that affect bike rentals.”

“After that, I selected the output value that I want the model to predict. Here, the target is the total number of bikes rented.”

“I applied a mathematical log function to the bike count so that very large values become easier for the model to understand.”

“Then, I created the input dataset by taking only the selected features from the main data.”

“So now, I have:

- Input data, which contains the conditions like weather and time
- Output data, which contains the number of bikes rented”

“Next, I divided the data into two parts.”

“80 percent of the data is used for training the model, and 20 percent is used for testing the model.”

“This helps me check whether the model works well on new data.”

“Finally, I printed the shape of the training and testing data.”

“The output shows that:

- 13,898 records are used for training
- 3,475 records are used for testing
- Each record contains 23 features”

“This code prepares data for machine learning by selecting important features, transforming the target value, and splitting the data into training and testing sets.”

```
# Model Evaluation Function
from sklearn.metrics import mean_squared_error, r2_score

def evaluate_model(name, model):
    preds = model.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, preds))
    r2 = r2_score(y_test, preds)
    print(f"{name}")
    print(f"RMSE : {rmse:.4f}")
    print(f"R² : {r2:.4f}")
    print("-"*30)
    return rmse, r2
```

“After training my machine learning model, I need to check how good my model is. This code helps me **evaluate the model’s performance**.”

```
from sklearn.metrics import mean_squared_error, r2_score
```

“First, I imported two tools used to measure how accurate the model is.”

- **Mean Squared Error** → tells how much mistake the model makes
- **R2 Score** → tells how well the model understands the data pattern

```
def evaluate_model(name, model):
```

“I created a function so I can evaluate **any model easily** by just giving its name and the model itself.”

👉 This avoids writing the same code again and again.

```
preds = model.predict(X_test)
```

“Here, the model predicts the bike rental count using the test data.”

📌 Important:

- Test data is **new data**
- The model has **never seen it before**

```
rmse = np.sqrt(mean_squared_error(y_test, preds))
```

“RMSE tells us how far the predicted value is from the actual value.”

Simple meaning:

- **Low RMSE** → good prediction
- **High RMSE** → bad prediction

👉 Like:

- Actual answer = 100
- Predicted = 98 → small mistake (good)

```
r2 = r2_score(y_test, preds)
```

“R² score tells how well the model understands the pattern in the data.”

Easy scale:

- **R² = 1** → perfect model
- **R² close to 0** → weak model

- **R² negative** → very poor model

```
print(f"{name}")
print(f"RMSE : {rmse:.4f}")
print(f"R2    : {r2:.4f}")
print("-"*30)
```

“This part prints the model name, its error value, and its accuracy score in a clean format.”

👉 This makes comparison between different models easy.

“Finally, the function returns RMSE and R² values so they can be stored or compared later.”

“This code checks how many mistakes the computer made and how smartly it learned from the data.”

“If the model predicts 500 bikes but actually 520 bikes were rented, RMSE tells how big that mistake is, and R² tells how well the model understands bike demand overall.”

“This function evaluates the trained model by calculating error and accuracy scores using test data.”

```
# Linear Regression (Baseline)
from sklearn.linear_model import LinearRegression

lr = LinearRegression()
lr.fit(X_train, y_train)

evaluate_model("Linear Regression", lr)

• Linear Regression
  RMSE : 0.7282
  R2   : 0.7371
  -----
  (np.float64(0.7281607833530608), 0.7371294229020502)
```

“In this part, I train a basic machine learning model called **Linear Regression** and check how well it predicts bike rentals.”

```
from sklearn.linear_model import LinearRegression
```

“Here, I import Linear Regression, which is a simple and commonly used machine learning model.”

👉 It works by finding a **straight-line relationship** between inputs and output.

```
lr = LinearRegression()
```

“I create an object of the Linear Regression model.”

```
lr.fit(X_train, y_train)
```

“Now I train the model using the training data.”

“The model learns how factors like weather, time, and season affect bike rentals.”

👉 This is like:

- Giving practice questions (X_{train})
- And correct answers (y_{train})

```
evaluate_model("Linear Regression", lr)
```

“After training, I test the model using unseen data to check its performance.”

“This step tells me how accurate the model is.”



Understanding the Output

Output shown:

```
Linear Regression  
RMSE : 0.7282  
R2    : 0.7371
```

“RMSE shows how much mistake the model makes on average.”

- **Lower RMSE = better prediction**
- RMSE = **0.7282** means the model’s prediction error is small

👉 So the model is doing a **good job**.

“ R^2 score shows how well the model understands the pattern in the data.”

- $R^2 = \mathbf{0.7371}$
- This means the model explains about **73% of the bike rental behavior**

👉 This is considered **good accuracy** for a simple model.

“I trained a simple model that learns from past data and predicts bike rentals. It makes some mistakes, but overall it understands most of the pattern.”

“If the model predicts 300 bikes and actual demand is 310 bikes, the error is small, so RMSE stays low.”

“Because the R^2 value is high, it means the model understands how weather and time affect bike rentals.”

```
# Ridge Regression
from sklearn.linear_model import Ridge

ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)

evaluate_model("Ridge Regression", ridge)
# Better stability than linear regression

Ridge Regression
RMSE : 0.7281
R2  : 0.7372
-----
(np.float64(0.7281234029317925), 0.7371564113389981)
```

“In this step, I trained a **Ridge Regression model**, which is an improved version of Linear Regression that gives more stable and reliable results.”

```
from sklearn.linear_model import Ridge
```

“Here, I import Ridge Regression from the machine learning library.”

👉 Ridge Regression is similar to Linear Regression but **controls over-learning**.

```
ridge = Ridge(alpha=1.0)
```

“I create the Ridge Regression model and set the value of `alpha`.”

“Alpha controls how strong the correction is.”

- Small alpha → behaves like Linear Regression
- Larger alpha → more stable but simpler model

```
ridge.fit(X_train, y_train)
```

“Now I train the Ridge model using the training data.”

“The model learns the relationship between weather, time, and bike rentals, but in a more balanced way.”

```
evaluate_model("Ridge Regression", ridge)
```

“After training, I evaluate the model using test data to check its performance.”

Output shown:

```
Ridge Regression  
RMSE : 0.7281  
R2    : 0.7372
```

“RMSE tells how much error the model makes.”

- RMSE = **0.7281**
- This is **slightly lower** than Linear Regression

👉 This means Ridge Regression makes **slightly fewer mistakes**.

“R² score tells how well the model understands the pattern in data.”

- R² = **0.7372**
- This means the model explains **about 73%** of bike rental behavior

👉 Very similar to Linear Regression but **more stable**.

“Ridge Regression prevents very large weights for features.”

“This helps the model avoid over-fitting and perform better on new data.”

Real-Life Example

“If temperature suddenly changes a lot, Linear Regression may react too strongly.”

“Ridge Regression keeps predictions more balanced and realistic.”

```
# Lasso Regression
from sklearn.linear_model import Lasso

lasso = Lasso(alpha=0.001)
lasso.fit(X_train, y_train)

evaluate_model("Lasso Regression", lasso)

• Lasso Regression
  RMSE : 0.7289
  R²   : 0.7366
  -----
  (np.float64(0.7289371796654378), 0.736568556244427)
```

“In this step, I trained a **Lasso Regression model**, which not only predicts bike rentals but also automatically removes less important features.”

```
from sklearn.linear_model import Lasso
```

“Here, I import Lasso Regression from the machine learning library.”

👉 Lasso Regression is similar to Linear and Ridge Regression but it can **ignore unnecessary inputs**.

```
lasso = Lasso(alpha=0.001)
```

“I create the Lasso Regression model and set the value of `alpha`.”

What does `alpha` mean?

“Alpha controls how strictly the model removes less useful features.”

- Small alpha → keeps more features
- Large alpha → removes more features

```
lasso.fit(X_train, y_train)
```

“Now I train the Lasso model using the training data.”

“The model learns which features are important and which can be ignored.”

```
evaluate_model("Lasso Regression", lasso)
```

“After training, I test the model using new data to check its accuracy.”

Output shown:

```
Lasso Regression
RMSE : 0.7289
```


R2 : 0.7366

“RMSE shows how much error the model makes.”

- RMSE = **0.7289**
- This is slightly **higher** than Linear and Ridge Regression

👉 This means the model makes **slightly more mistakes**.

“R² score tells how well the model understands the pattern in the data.”

- R² = **0.7366**
- This means the model explains about **73%** of the bike rental behavior

“Lasso Regression removes some features by making their importance zero.”

“This simplifies the model but can reduce accuracy if useful features are removed.”

“If humidity has very little effect on bike rentals, Lasso Regression may completely ignore it.”

“This makes the model simpler but sometimes less accurate.”

```
# Random Forest Regressor
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(
    n_estimators=200,
    random_state=42
)
rf.fit(X_train, y_train)

evaluate_model("Random Forest", rf)

Random Forest
RMSE : 0.3256
R2 : 0.9474
-----
(np.float64(0.32558259458451405), 0.9474455170920252)
```

“In this step, I trained a **Random Forest model**, which is a powerful machine learning model that combines many decision trees to give very accurate predictions.”

```
from sklearn.ensemble import RandomForestRegressor
```

“Here, I import the Random Forest Regressor from the machine learning library.”

👉 Random Forest works by using **many decision trees together**.

```
rf = RandomForestRegressor(  
    n_estimators=200,  
    random_state=42  
)
```

“I create the Random Forest model with 200 decision trees.”

```
n_estimators = 200
```

“It means the model uses **200 trees** and takes the average of their predictions.”

More trees → better learning → more accuracy.

```
random_state = 42
```

“It makes the result the same every time we run the code.”

```
rf.fit(X_train, y_train)
```

“Now, I train the Random Forest model using training data.”

“Each tree learns different patterns from the data.”

👉 Together, they make a **strong and accurate model**.

```
evaluate_model("Random Forest", rf)
```

“After training, I evaluate the model using unseen test data.”

Output shown:

```
Random Forest  
RMSE : 0.3256  
R2   : 0.9474
```

“RMSE shows how much error the model makes.”

- RMSE = **0.3256**
- This is **much lower** than previous models

👉 This means the model makes **very small mistakes**.

“R² score tells how well the model understands the data.”

- R² = **0.9474**

- This means the model explains **about 95% of bike rental behavior**

👉 This is **excellent accuracy**.

“Random Forest can learn **complex relationships** between features.”

“It does not depend on a straight line like Linear Regression.”

“Because many trees work together, errors are reduced.”

example

“One tree may focus on weather, another on time, and another on season.”

“When all of them give their opinion together, the final answer becomes very accurate.”

“Random Forest is clearly the best model in this project.”

```
# Gradient Boosting Regressor
from sklearn.ensemble import GradientBoostingRegressor

gbr = GradientBoostingRegressor()
gbr.fit(X_train, y_train)

evaluate_model("Gradient Boosting", gbr)

Gradient Boosting
RMSE : 0.4027
R²   : 0.9196
-----
(np.float64(0.40273797322181143), 0.9195858328846749)
```

“In this step, I trained a **Gradient Boosting model**, which improves predictions by learning from its previous mistakes.”

```
from sklearn.ensemble import GradientBoostingRegressor
```

“Here, I import the Gradient Boosting Regressor from the machine learning library.”

👉 This model works by **adding small models one by one**.

```
gbr = GradientBoostingRegressor()
```

“I create the Gradient Boosting model using default settings.”

👉 Each small model focuses on **correcting the errors** made by the previous one.

```
gbr.fit(X_train, y_train)
```

“Now I train the model using training data.”

“The model learns step-by-step, improving itself every time.”

```
evaluate_model("Gradient Boosting", gbr)
```

“After training, I test the model using unseen data to check its accuracy.”

Output shown:

```
Gradient Boosting  
RMSE : 0.4027  
R2    : 0.9196
```

“RMSE tells how much error the model makes.”

- RMSE = **0.4027**
- This is **much lower than linear models**
- Slightly higher than Random Forest

👉 This means the model predicts **very accurately**.

“R² score shows how well the model understands the data.”

- R² = **0.9196**
- This means the model explains **about 92%** of bike rental behavior

👉 This is **excellent performance**.

Real-Life Example

“If the model predicts 400 bikes but actual demand is 420, the next model learns from this error and improves the next prediction.”

“Gradient Boosting is a powerful model that improves accuracy by correcting its own mistakes step by step.”

```
# XGBoost (BEST MODEL)
from xgboost import XGBRegressor

xgb = XGBRegressor(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)

xgb.fit(X_train, y_train)

evaluate_model("XGBoost", xgb)

• XGBoost
  RMSE : 0.2803
  R²   : 0.9610
  -----
  (np.float64(0.28032833553969183), 0.9610397796387238)
```

“In this step, I trained an **XGBoost model**, which is the most advanced and best-performing model in my project.”

```
from xgboost import XGBRegressor
```

“Here, I import XGBoost Regression from the XGBoost library.”

👉 XGBoost is an **improved version of Gradient Boosting**, designed for **high accuracy and speed**.

```
xgb = XGBRegressor(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)
```

“I create the XGBoost model and carefully set important parameters.”

- **n_estimators = 300**
→ The model uses **300 trees** to learn patterns
- **learning_rate = 0.05**
→ The model learns **slowly and carefully** to avoid mistakes

- **max_depth = 6**
→ Limits how complex each tree can be
- **subsample = 0.8**
→ Uses only 80% of data at a time to reduce over-learning
- **colsample_bytree = 0.8**
→ Uses only 80% of features per tree
- **random_state = 42**
→ Ensures same results every time

```
xgb.fit(X_train, y_train)
```

“Now I train the XGBoost model using training data.”

“The model learns step-by-step and corrects its mistakes very efficiently.”

```
evaluate_model("XGBoost", xgb)
```

“After training, I test the model on new data to check how accurate it is.”

Output shown:

```
XGBoost
RMSE : 0.2803
R2   : 0.9610
```

“RMSE tells how much error the model makes.”

- RMSE = **0.2803**
- This is the **lowest error among all models**

👉 This means XGBoost makes **very small mistakes**.

“R² score tells how well the model understands the data.”

- R² = **0.9610**
- This means the model explains **96% of bike rental behavior**

👉 This is **excellent accuracy**.

◆ Why XGBoost is the BEST model?

“XGBoost learns from mistakes like Gradient Boosting but in a faster and smarter way.”

“It controls over-fitting, handles complex patterns, and gives very accurate results.”

example

“If the model predicts 500 bikes and the actual demand is 505, the error is very small.”

“That’s why RMSE is low and accuracy is high.”

“XGBoost is the best model in my project because it gives the highest accuracy and the lowest prediction error.”

Final

```
Lasso
RMSE : 0.7289
R² : 0.7366
-----
Random Forest
RMSE : 0.3256
R² : 0.9474
-----
Gradient Boosting
RMSE : 0.4027
R² : 0.9196
-----
XGBoost
RMSE : 0.2803
R² : 0.9610
-----
```

	Model	RMSE	R2
5	XGBoost	0.280328	0.961040
3	Random Forest	0.325583	0.947446
4	Gradient Boosting	0.402738	0.919586
1	Ridge	0.728123	0.737156
0	Linear	0.728161	0.737129
2	Lasso	0.728937	0.736569

“In this step, I compared all the machine learning models to find which one performs the best.”

```
results = []
```

“I create an empty list to store the results of each model.”

👉 This list will save:

- Model name
- Error value
- Accuracy score

```
models = {
    "Linear": lr,
    "Ridge": ridge,
```

```

    "Lasso": lasso,
    "Random Forest": rf,
    "Gradient Boosting": gbr,
    "XGBoost": xgb
}

```

“Here, I store all trained models in one place using a dictionary.”

👉 This makes it easy to test all models one by one.

```

for name, model in models.items():
    rmse, r2 = evaluate_model(name, model)
    results.append([name, rmse, r2])

```

“I use a loop to evaluate every model.”

“For each model:

- I calculate error (RMSE)
- I calculate accuracy (R^2)
- I store the results”

```

results_df = pd.DataFrame(results, columns=["Model", "RMSE", "R2"])

```

“I convert the results into a table format so it is easy to compare.”

```

results_df.sort_values("R2", ascending=False)

```

“I sort the table by R^2 score in descending order.”

👉 Best model comes **on top**.

Final sorted table:

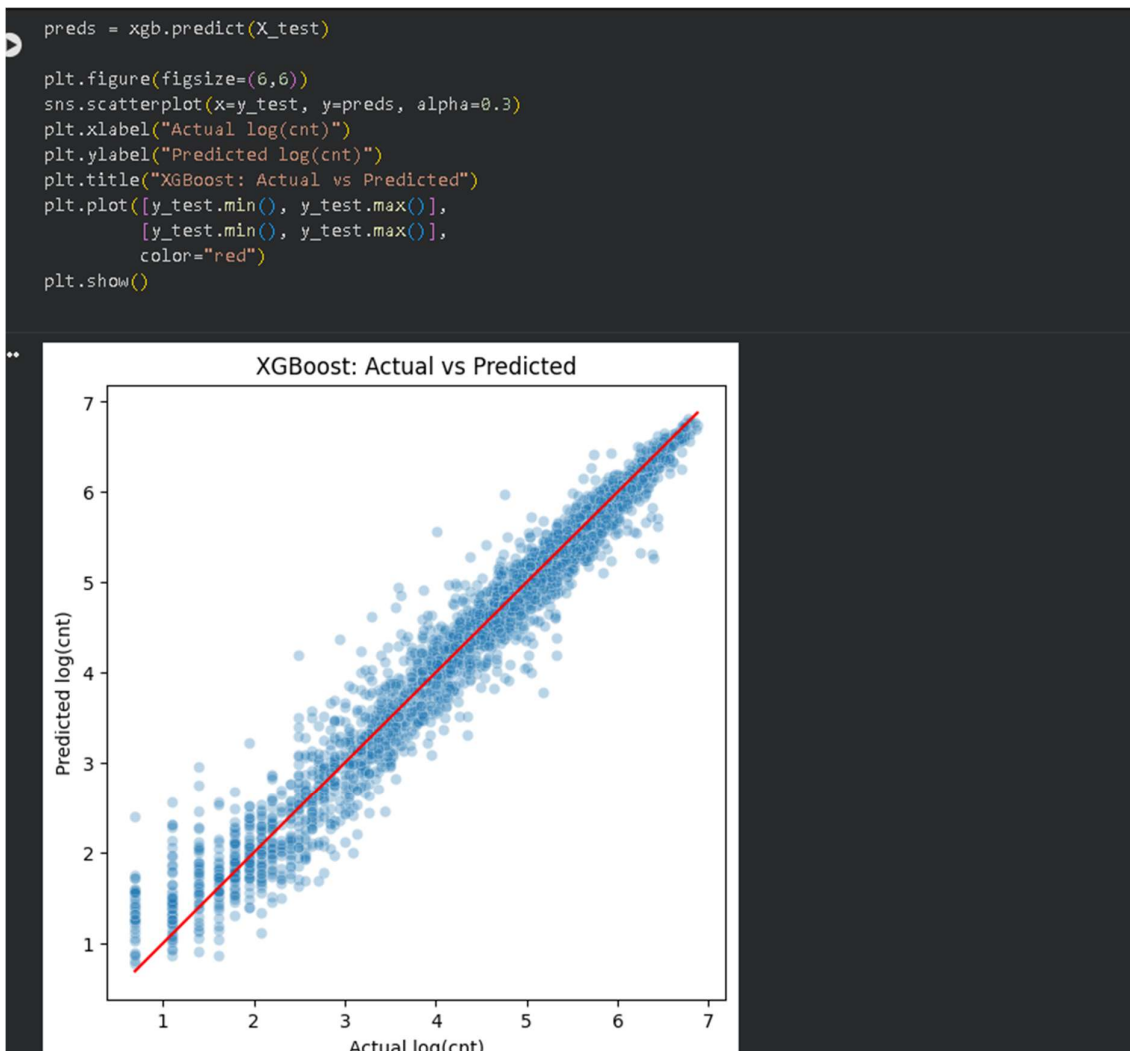
Rank	Model	RMSE	R^2
1	XGBoost	0.2803	0.9610
2	Random Forest	0.3256	0.9474
3	Gradient Boosting	0.4027	0.9196
4	Ridge	0.7281	0.7372
5	Linear	0.7282	0.7371
6	Lasso	0.7289	0.7366

“The model with the **lowest RMSE** and **highest R^2** is the best model.”

- **XGBoost** has:
 - Lowest error
 - Highest accuracy

👉 That is why **XGBoost** is selected as the final model.

“After comparing all models, XGBoost gave the highest accuracy and lowest error, so it is chosen as the final model.”



“In this step, I visually check how close my model’s predictions are to the actual bike rental values.”

```
preds = xgb.predict(X_test)
```

“Here, the XGBoost model predicts bike rental values using the test data.”

👉 These are the model's **final answers**.

```
plt.figure(figsize=(6,6))
sns.scatterplot(x=y_test, y=preds, alpha=0.3)
```

“I create a scatter plot where:

- X-axis shows **actual bike rentals**
- Y-axis shows **predicted bike rentals**”

“Each dot represents one data point.”

```
plt.xlabel("Actual log(cnt)")
plt.ylabel("Predicted log(cnt)")
plt.title("XGBoost: Actual vs Predicted")
```

“I label the graph so it is easy to understand what is being compared.”

◆ Step 4: Drawing the red diagonal line

```
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    color="red"
)
```

“This red line represents **perfect prediction**.”

👉 If a point lies:

- **On the red line** → prediction is perfect
- **Close to the red line** → prediction is very good
- **Far from the red line** → prediction is poor

```
plt.show()
```

“Finally, the graph is displayed.”

“Most of the blue points are very close to the red line.”

👉 This means:

- Predicted values $\approx$ Actual values
- Errors are very small
- Model is **highly accurate**

“This graph shows how close the computer’s answers are to the real answers.”

“Because most dots are near the red line, the computer is predicting very correctly.”

Real-Life Example

“If actual bike rentals are 500 and the model predicts 495 or 505, the dot will be very close to the red line.”

“That shows the model is reliable.”

Final Summary Line (Mentor-Ready)

“This graph proves that XGBoost predicts bike rentals very accurately because the predicted values closely match the actual values.”